

Evoke AI Search Visibility Audit

Cursor Build & Product Specification

62-parameter backend audit engine + 2-page web application + downloadable final report

Working example: All examples use <https://www.evoketechnologies.com>. The baseline figures in the source audit are used only as an example of how the engine should report findings; the scanner must re-fetch live data during each scan.

0. Product Requirement — What You Are Building

Build a production-style AI Search Visibility Audit web application for a target website. The initial showcase target is <https://www.evoketechnologies.com/>. The user enters a website URL, clicks Submit, and the backend launches an independent scan pipeline that evaluates exactly 62 core parameters: 22 Technical, 22 On-Page/Content, and 18 Off-Page/Authority. Each parameter must execute independently, produce its own score/status/evidence/recommendation, and contribute to category and overall scores.

0.1 Required User Experience

Page 1 — Scan/Input: show a clean professional landing screen with the product name, a single URL input prefilled with <https://www.evoketechnologies.com/>, and a Submit / Start Audit action. After submission, create a scan job and show a progress state that identifies the 3 section pipelines and their progress: Technical (22/22), On-Page (22/22), Off-Page (18/18). The UI must not fabricate scores while the scan is running.

Page 2 — Report: after the backend finishes, navigate to a professional audit report. Show the overall score, Technical/On-Page/Off-Page category scores, parameter status counts, priority findings, and drill-downs to individual parameter evidence. Include a visible Download Report icon/button that exports the completed report to PDF. The report must be based only on stored scan results, not client-side recalculation.

0.2 Backend Execution Requirement

When Submit is clicked, the frontend calls POST /api/scans. The backend creates scan_id and executes the 62 checks through section-specific services. Technical and On-Page checks should run concurrently where dependencies permit; Off-Page checks should run through an adapter layer. Each parameter must have an independent status so one failed check does not stop the remaining 61. The orchestrator records started/completed/error state for every parameter and aggregates only after all runnable checks finish.

0.3 Core APIs for Cursor

POST /api/scans
GET /api/scans/{scan_id}
GET /api/scans/{scan_id}/progress
GET /api/scans/{scan_id}/report
GET /api/scans/{scan_id}/parameters
GET /api/scans/{scan_id}/parameters/{parameter_id}
GET /api/scans/{scan_id}/download

The scan status endpoint should return: scan_id, domain, status, progress_percent, technical_completed/total, onpage_completed/total, offpage_completed/total, started_at, completed_at, errors_count. The report endpoint returns persisted scores and findings.

0.4 Required Result Contract

```
parameter_result = {
  scan_id, parameter_id, section, name, status, score, max_score,
  weight, confidence, checked_url_or_source, evidence,
  recommendation, error, duration_ms, evaluated_at
}
```

0.5 Non-Negotiable Rules

Do not create placeholder PASS values. Do not infer a live result from the uploaded PDF. The PDF is the product-reference baseline; the new application must fetch live data at scan time. If a source cannot be accessed, return UNKNOWN with the exact reason. Every finding must include evidence sufficient for a user to understand why the status was assigned. Keep raw source snapshots or normalized evidence hashes so results are auditable.

0.6 Scope Boundary

The 62-parameter core scope is intentionally limited to Technical, On-Page/Content, and Off-Page/Authority. AI Visibility by assistant and Market Consistency are separate future modules from the uploaded audit and should not be silently counted inside the 62 unless explicitly added as new parameters.

1. Executive Summary

Recommended core audit scope: 62 parameters: 22 Technical, 22 On-Page/Content, and 18 Off-Page/Authority. This is a practical product scope, not a claim that the web has only 62 SEO/AI-visibility signals.

Section	Parameters	Primary evidence	Automation level
Technical	22	robots.txt, HTTP, sitemap, HTML, headers, JSON-LD, performance tooling	High
On-Page / Content	22	HTML, headings, links, text, schema, NLP/LLM evaluation	High to Medium
Off-Page / Authority	18	Backlink/review/directory/search data, external profiles, mention discovery	Medium to Low without APIs
Total	62	Combined evidence + scoring engine	Phased

Source baseline: The uploaded audit reports an overall score of 69/100, Technical Health 89/100, Content & Answers 60/100, and Reputation & Authority 54/100. Its weighted score is 35% Technical, 40% Content & Answers, and 25% Reputation & Authority. The source also reports 201 sitemap URLs, a 23/100 performance estimate, 60% image alt-text coverage, and 43 lists/tables. Use these only as baseline reference points; your new engine should calculate its own live results.

2. Recommended System Architecture

User enters domain -> Scan Orchestrator -> URL Discovery -> Technical Crawler / Content Analyzer / Off-Page Adapters -> Parameter Engine -> Scoring Engine -> Findings & Recommendations -> Dashboard / Export

Component	Responsibility	Recommended implementation
Scan Orchestrator	Creates scan, controls stages, retries, timeouts	FastAPI/Node API + background worker
URL Discovery	Finds canonical pages from sitemap,	Python requests/httpx +

	robots, internal links	BeautifulSoup/lxml
Technical Crawler	HTTP status, redirects, headers, HTML, schema, images, links	httpx/requests + Playwright where JS is needed
Performance Runner	Core Web Vitals / Lighthouse style data	Lighthouse CLI or PageSpeed API where permitted
Content Analyzer	Question headings, definitions, answers, FAQs, buyer journey	BeautifulSoup + NLP + optional LLM evaluator
Off-Page Adapter Layer	Backlinks, profiles, reviews, mentions, directories	Provider-agnostic API adapters
Parameter Engine	Runs deterministic and AI rules	JSON/YAML parameter registry + Python
Scoring Engine	Weighting, category score, overall score	Pure Python service with test coverage
Evidence Store	Raw inputs, extracted evidence, timestamps	PostgreSQL or SQLite for POC; object storage for large HTML
Dashboard	Overview, findings, evidence, fixes, trends	React/Next.js or Streamlit for POC

3. Core Data Model

scan(id, domain, started_at, completed_at, status, crawler_version)

page(scan_id, url, final_url, status_code, content_type, response_time_ms, html_hash, canonical_url, word_count)

parameter_result(scan_id, parameter_id, score, status, weight, evidence_json, recommendation, confidence)

external_source(scan_id, source_name, entity, retrieved_at, payload_hash, status)

issue(scan_id, issue_id, severity, affected_urls, effort, recommendation)

Field	Purpose
parameter_id	Stable ID, e.g. TECH-001
status	PASS / PARTIAL / FAIL / UNKNOWN
score	0-100 normalized score
weight	Importance within category
evidence	Machine-readable and human-readable proof
confidence	0-1 confidence, especially for AI/external signals
source	URL/API/page/document that produced evidence
checked_at	Timestamp of evidence retrieval
recommendation	Actionable remediation text

4. Scoring Framework

Category Score = $\Sigma(\text{parameter_score} \times \text{parameter_weight}) / \Sigma(\text{parameter_max} \times \text{parameter_weight}) \times 100$

Overall Score = (Technical $\times$ 0.35) + (On-Page $\times$ 0.40) + (Off-Page $\times$ 0.25)

Status	Default score mapping	Meaning
PASS	90-100	Meets the defined acceptance rule
PARTIAL	60-89	Some coverage or quality, but below target
FAIL	0-59	Does not meet the rule
UNKNOWN	Not scored	Required data unavailable or integration not connected

Important scoring rule: Do not convert UNKNOWN to 0. Keep it separate so a missing API does not falsely lower the client score. For overall reporting, either exclude UNKNOWN parameters from the denominator or

publish a second “coverage” metric. Choose one policy and keep it consistent.

5. Technical Parameters — 22

TECH-001 — AI crawler permissions

Field	Implementation specification
Source / URL / input	https://www.evoketechnologies.com/robots.txt
Why it matters	Checks whether major AI crawler user-agents are explicitly blocked or allowed.
Exact implementation steps	Fetch robots.txt with GET; parse User-agent groups; evaluate GPTBot, ChatGPT-User, ClaudeBot, Google-Extended, PerplexityBot; apply wildcard group when no specific group exists.
Decision logic	If a bot matches Disallow: / it is blocked. If relevant paths are allowed, mark allowed/partial based on scope.
Calculation	Allowed crawler score = allowed crawlers / crawlers checked × 100.
Status rules	PASS >=90; PARTIAL 60-89; FAIL <60; UNKNOWN if robots.txt cannot be retrieved.
Evidence to store	robots URL, HTTP status, raw rules, matched rule per bot, parsed decision, scan timestamp.
Recommendation	Show blocked bots and the exact matched rule; recommend the smallest rule change that permits desired public content.
Implementation tools	Python httpx/requests + robots.txt parser

Evoke example: The source audit reports AI crawler permissions as Pass for the audited scan. Re-fetch live data for the new scanner.

Caveat: Robots rules are crawl directives, not a guarantee that an AI service will use or quote the content.

TECH-002 — AI crawler accessibility

Field	Implementation specification
Source / URL / input	Homepage and a representative set of important pages on the target domain
Why it matters	Checks whether AI crawler-like HTTP requests can actually reach pages, not just robots permission.
Exact implementation steps	Select homepage + top sitemap/service URLs; request them with normal browser and crawler user-agents; record status, redirects, content type and blocking responses.
Decision logic	2xx with HTML and no bot-specific block = allowed. 403/429/5xx or challenge page only for crawler UA = blocked/partial.
Calculation	Accessible test URLs / total test URLs × 100.
Status rules	PASS >=90; PARTIAL 60-89; FAIL <60; UNKNOWN if test cannot run.
Evidence to store	URL, UA, status, final URL, headers, response size, challenge markers.
Recommendation	Identify bot-specific WAF/CDN blocks and exempt verified crawlers where appropriate.

Implementation tools	httpx/requests; optional Playwright for challenge pages
----------------------	---

Evoke example: The source audit reports no obvious CDN/firewall rules silently blocking AI crawlers.

Caveat: Do not label a page blocked from one transient 5xx; retry before failure.

TECH-003 — llms.txt availability

Field	Implementation specification
Source / URL / input	https://www.evoketechnologies.com/llms.txt
Why it matters	Checks whether an llms.txt file exists and contains useful, parseable guidance/content.
Exact implementation steps	GET /llms.txt; check status; capture content; validate basic structure; measure substantive content.
Decision logic	404/410 = absent. 200 + non-empty useful content = present. 200 + empty/garbage = partial.
Calculation	Existence score + content-quality subscore; simple V1: 100 if 200 and substantive, 50 if 200 but weak, 0 if absent.
Status rules	PASS 90-100; PARTIAL 50-89; FAIL 0-49; UNKNOWN on retrieval failure.
Evidence to store	URL, status, content hash, byte count, headings/links, validator results.
Recommendation	Create or update llms.txt with stable descriptions and priority pages if absent.
Implementation tools	httpx + Markdown parser

Evoke example: The source audit reports llms.txt present with substantive content.

Caveat: Treat llms.txt as an optional signal; do not let it dominate the total score.

TECH-004 — Login, pop-up or cookie wall blocking

Field	Implementation specification
Source / URL / input	Homepage and primary content URLs
Why it matters	Checks whether the main content is obscured by login, consent, or interstitial overlays.
Exact implementation steps	Fetch raw HTML; optionally render with Playwright; detect modal/overlay elements, login forms, paywall markers and low content visibility.
Decision logic	If main content is available without interaction, pass. If a gate covers primary content, fail/partial depending on scope.
Calculation	Accessible primary-content pages / tested pages × 100.
Status rules	PASS ≥90; PARTIAL 60-89; FAIL <60.
Evidence to store	Screenshot hash/DOM selector, detected marker, URL, rendered text availability.
Recommendation	Remove or reduce intrusive gating over content intended for discovery.
Implementation tools	BeautifulSoup + Playwright + heuristic classifiers

Evoke example: The source audit reports no login, pop-up or cookie wall covering main content.

Caveat: Cookie consent banners vary by region; test both raw HTML and rendered state.

TECH-005 — XML sitemap completeness

Field	Implementation specification
Source / URL / input	https://www.evoketechnologies.com/sitemap.xml or robots-discovered sitemap
Why it matters	Checks whether the sitemap is reachable, parseable, internally consistent and representative of indexable URLs.
Exact implementation steps	Read robots.txt for Sitemap entries; GET each sitemap; parse sitemap index recursively; validate URLs; sample URLs; compare to discovered internal canonical URLs.
Decision logic	Invalid XML or broken sitemap = fail. Missing important canonical pages from sitemap = partial.
Calculation	Valid sitemap URLs / sitemap URLs checked, combined with coverage ratio.
Status rules	PASS >=90; PARTIAL 60-89; FAIL <60.
Evidence to store	Sitemap URLs, parse errors, sampled status codes, canonical mismatch counts, total URL count.
Recommendation	Fix invalid entries, add important indexable pages, remove stale/non-canonical URLs.
Implementation tools	lxml + xml.etree + crawler

Evoke example: The source audit reports a sitemap with 201 URLs.

Caveat: Sitemap count is not the same as indexed count; report them separately.

TECH-006 — Broken internal links

Field	Implementation specification
Source / URL / input	All internally linked HTML pages within crawl scope
Why it matters	Checks for internal links returning error responses.
Exact implementation steps	Discover internal links from pages; canonicalize URLs; de-duplicate; request each URL with bounded concurrency; classify 2xx/3xx/4xx/5xx.
Decision logic	4xx/5xx are broken candidates. 3xx is a redirect, not automatically broken.
Calculation	Working internal URLs / tested internal URLs × 100.
Status rules	PASS >=98; PARTIAL 90-97; FAIL <90.
Evidence to store	Source page, target URL, status, redirect chain, error body.
Recommendation	Fix 4xx/5xx links and unnecessary redirects from high-value pages.
Implementation tools	httpx + BeautifulSoup/lxml

Evoke example: The source audit sampled 5 internal links and found none returning errors.

Caveat: Use a crawl limit and prioritize HTML pages to control scan time.

TECH-007 — Redirect chains and loops

Field	Implementation specification
Source / URL / input	Every discovered internal URL plus homepage variants
Why it matters	Checks unnecessary multi-hop redirects and redirect loops.
Exact implementation steps	Request with redirects disabled; follow Location headers manually up to N hops; record chain.
Decision logic	0-1 redirect = healthy. 2+ redirects = chain. Repeated URL = loop.
Calculation	Healthy redirect ratio = URLs with ≤ 1 redirect / redirects tested $\times 100$.
Status rules	PASS ≥ 95 ; PARTIAL 85-94; FAIL < 85 .
Evidence to store	Original URL, each hop, status, final URL, loop flag.
Recommendation	Replace chains with direct destination links and canonical references.
Implementation tools	httpx/requests with allow_redirects=False

Evoke example: Use 10-hop maximum to avoid loops.

Caveat: Do not treat a single HTTPS or www normalization redirect as a severe issue.

TECH-008 — Page speed / performance score

Field	Implementation specification
Source / URL / input	Representative pages + Lighthouse/PageSpeed tooling
Why it matters	Measures load performance under a consistent test profile.
Exact implementation steps	Run Lighthouse or equivalent on homepage and representative templates; collect performance score and key metrics.
Decision logic	Use tool score as reported; keep raw metrics. For a custom score, normalize LCP/INP/CLS/TTFB.
Calculation	Performance score = weighted normalized submetrics.
Status rules	PASS ≥ 90 ; PARTIAL 60-89; FAIL < 60 .
Evidence to store	Device, network profile, performance score, raw metrics, tool version, run timestamp.
Recommendation	Optimize render-blocking resources, images, JS, server response and caching.
Implementation tools	Lighthouse CLI or PageSpeed API; Playwright optional

Evoke example: The source audit estimated performance at 23/100 because PageSpeed was unavailable at the time.

Caveat: Performance is variable; run multiple times and use median/representative result.

TECH-009 — LCP

Field	Implementation specification
Source / URL / input	Representative page set
Why it matters	Checks Largest Contentful Paint.
Exact implementation steps	Run browser-based performance trace; extract LCP per page; use median across runs/pages.
Decision logic	Good ≤ 2.5 s; needs improvement 2.5-4s; poor > 4 s

	(implementation thresholds can be configurable).
Calculation	Convert each page to 100-point score: $\leq 2.5=100$, $4+=0$, interpolate between.
Status rules	PASS ≥ 90 ; PARTIAL 60-89; FAIL <60 .
Evidence to store	LCP value, element selector, page, device, run ID.
Recommendation	Optimize hero image, server response, CSS, fonts and render-blocking assets.
Implementation tools	Lighthouse/Playwright Performance API

Evoke example: Use thresholds as configuration so they can be updated without code changes.

Caveat: CWV thresholds are external standards and should be versioned in the ruleset.

TECH-010 — INP

Field	Implementation specification
Source / URL / input	Representative interactive pages
Why it matters	Checks responsiveness of real or synthetic interactions.
Exact implementation steps	Use browser performance tooling to capture interaction latency; collect INP or closest supported metric.
Decision logic	Use configurable thresholds; aggregate by page template.
Calculation	Normalized score based on observed INP.
Status rules	PASS/PARTIAL/FAIL from configured threshold bands.
Evidence to store	INP, interaction type, page, environment.
Recommendation	Reduce long tasks, heavy JS and main-thread contention.
Implementation tools	Lighthouse/CrUX/PageSpeed where available

Evoke example: If field data is unavailable, label the result synthetic rather than field.

TECH-011 — CLS

Field	Implementation specification
Source / URL / input	Representative pages
Why it matters	Checks visual stability.
Exact implementation steps	Run layout shift trace; sum session-window shifts; record CLS.
Decision logic	Good ≤ 0.1 ; needs improvement 0.1-0.25; poor >0.25 (configurable).
Calculation	Normalize to 0-100 based on threshold bands.
Status rules	PASS/PARTIAL/FAIL from CLS thresholds.
Evidence to store	CLS score and major shifting elements.
Recommendation	Reserve image/ad space, stabilize fonts, avoid injected layout shifts.
Implementation tools	Lighthouse/Playwright

Evoke example: Keep raw value in evidence.

TECH-012 — Mobile viewport

Field	Implementation specification
Source / URL / input	All HTML pages or representative template pages

Why it matters	Checks whether pages declare a mobile viewport.
Exact implementation steps	Parse <meta name="viewport">.
Decision logic	If content contains width=device-width or equivalent mobile viewport, pass.
Calculation	Pages with valid viewport / pages checked ×100.
Status rules	PASS 100 or configured >=95; partial/fail below.
Evidence to store	URL and exact viewport tag.
Recommendation	Add a standard responsive viewport declaration.
Implementation tools	BeautifulSoup/lxml

Evoke example: Source audit reports HTTPS and mobile viewport as passing.

Caveat: A viewport tag alone does not prove responsive quality.

TECH-013 — HTTPS delivery

Field	Implementation specification
Source / URL / input	Domain variants and sampled pages
Why it matters	Checks HTTPS availability, HTTP->HTTPS normalization and mixed-content indicators.
Exact implementation steps	Request http://domain and https://domain; inspect final URL and SSL errors; inspect HTML for http:// subresources where practical.
Decision logic	HTTPS final URL and valid certificate = pass. HTTP remains primary = fail. Mixed active content = partial.
Calculation	HTTPS-compliant checks / total HTTPS checks ×100.
Status rules	PASS >=95; PARTIAL 80-94; FAIL <80.
Evidence to store	URL variants, final URLs, certificate errors, mixed-content URLs.
Recommendation	Force HTTPS and remove mixed-content dependencies.
Implementation tools	httpx/requests + TLS validation + browser console

Evoke example: Source audit reports HTTPS delivery as pass.

TECH-014 — Main content present in raw HTML

Field	Implementation specification
Source / URL / input	Representative content pages
Why it matters	Checks whether substantial content exists without relying entirely on client-side JS.
Exact implementation steps	Fetch raw HTML; extract visible text; render page with Playwright; compare raw text to rendered text.
Decision logic	High raw-content coverage = pass. Raw near-empty while rendered content is large = fail/partial.
Calculation	Raw meaningful words / rendered meaningful words ×100.
Status rules	PASS >=80; PARTIAL 40-79; FAIL <40.
Evidence to store	Raw word count, rendered word count, ratio, sample HTML.
Recommendation	Server-render critical content or provide crawlable HTML.
Implementation tools	httpx + BeautifulSoup + Playwright

Evoke example: Source audit reports substantial content in raw page source.

Caveat: Dynamic pages may legitimately render components client-side; judge content that matters for discovery.

TECH-015 — One H1 per page

Field	Implementation specification
Source / URL / input	All indexable HTML pages
Why it matters	Checks H1 count.
Exact implementation steps	Extract H1 elements from each page.
Decision logic	Exactly 1 = pass; 0 or >1 = partial/fail depending on severity.
Calculation	Pages with exactly one H1 / pages checked ×100.
Status rules	PASS ≥95; PARTIAL 80-94; FAIL <80.
Evidence to store	URL, H1 text, count.
Recommendation	Add one descriptive H1 and remove duplicate template H1s.
Implementation tools	BeautifulSoup/lxml

Evoke example: Source audit reports exactly one H1 and no skipped heading levels.

TECH-016 — Heading hierarchy

Field	Implementation specification
Source / URL / input	All indexable HTML pages
Why it matters	Checks heading levels for skips and structural order.
Exact implementation steps	Extract H1-H6 sequence; flag H3 after H1, H4 after H2, duplicate structural jumps.
Decision logic	No downward skip >1 level = pass.
Calculation	Pages without major skips / pages checked ×100.
Status rules	PASS ≥95; PARTIAL 85-94; FAIL <85.
Evidence to store	Heading sequence per page, first offending transition.
Recommendation	Fix heading nesting to reflect document hierarchy.
Implementation tools	BeautifulSoup/lxml

Evoke example: Keep decorative headings separate from semantic heading tags.

TECH-017 — Heading frequency / liftable passages

Field	Implementation specification
Source / URL / input	Content pages
Why it matters	Checks whether sections are small enough to be lifted as clean passages.
Exact implementation steps	Count words between consecutive headings; exclude boilerplate; calculate median and overlong section ratio.
Decision logic	Source audit uses ~379 words per heading as an example. Configure target range per page type.
Calculation	Score = percentage of content sections within configured word range.
Status rules	PASS ≥80; PARTIAL 60-79; FAIL <60.
Evidence to store	Words per heading, distribution, offending sections.
Recommendation	Split oversized sections with question-led subheadings.
Implementation tools	BeautifulSoup/lxml + tokenizer

Evoke example: Source audit flags heading frequency as partial at ~379 words per heading.

Caveat: Use page-type specific ranges; do not force tiny sections on long-form content.

TECH-018 — Tables and lists as real markup

Field	Implementation specification
Source / URL / input	All content pages
Why it matters	Checks whether lists/tables exist as semantic HTML instead of images.
Exact implementation steps	Count <code></code> , <code></code> , <code><table></code> ; optionally detect screenshot/table-image patterns.
Decision logic	Semantic structures = good. Tables embedded as images where a data table is expected = issue.
Calculation	Relevant pages with semantic list/table markup / relevant pages ×100.
Status rules	PASS ≥90; PARTIAL 60-89; FAIL <60.
Evidence to store	Counts and selectors, image candidates.
Recommendation	Convert important information into real HTML lists/tables.
Implementation tools	BeautifulSoup/lxml + OCR-free image heuristics

Evoke example: Source audit found 0 tables and 43 lists as real markup.

TECH-019 — Text-to-code ratio

Field	Implementation specification
Source / URL / input	Representative pages
Why it matters	Checks how much page HTML is meaningful text versus markup/scripts.
Exact implementation steps	Download raw HTML; remove script/style/noscript; count meaningful text characters and total HTML characters.
Decision logic	Text ratio = visible text chars / HTML chars ×100.
Calculation	Use bands calibrated from site/template baseline; simple V1: ≥30 good, 15-29 partial, <15 poor.
Status rules	PASS ≥30; PARTIAL 15-29; FAIL <15.
Evidence to store	Raw HTML bytes/chars, text chars, ratio.
Recommendation	Reduce excessive markup, duplicate widgets and script-heavy rendering where it harms crawlability.
Implementation tools	BeautifulSoup/lxml

Evoke example: Source audit reports 49.7% text-to-code ratio.

Caveat: Ratio is diagnostic, not a universal ranking factor.

TECH-020 — Image alt text

Field	Implementation specification
Source / URL / input	All image elements on HTML pages
Why it matters	Checks presence and quality of alt text for informative images.
Exact implementation steps	Extract <code></code> ; ignore decorative images only when

	explicitly marked decorative; inspect alt attribute length/content.
Decision logic	Image with meaningful alt = pass; missing/empty alt on informative image = fail.
Calculation	Informative images with valid alt / informative images ×100.
Status rules	PASS ≥90; PARTIAL 70-89; FAIL <70.
Evidence to store	Image URL/srcset, alt text, decorative flag, page.
Recommendation	Add concise descriptive alt text to informative images.
Implementation tools	BeautifulSoup/lxml

Evoke example: Source audit reports 60% of images have alt text.

Caveat: Alt text should describe the image, not stuff keywords.

TECH-021 — Published and updated date signals

Field	Implementation specification
Source / URL / input	Article/service/resource pages
Why it matters	Checks whether a meaningful publication/update date is visible and/or marked up.
Exact implementation steps	Inspect <time>, meta datePublished/dateModified, JSON-LD Article fields, visible page text.
Decision logic	At least one trustworthy date signal = pass; inconsistent future/invalid dates = fail/partial.
Calculation	Pages with valid date signal / eligible pages ×100.
Status rules	PASS ≥90; PARTIAL 70-89; FAIL <70.
Evidence to store	Date source field, parsed date, schema field, consistency flags.
Recommendation	Add visible and schema-backed publication/update dates where appropriate.
Implementation tools	BeautifulSoup/lxml + date parser + JSON-LD parser

Evoke example: Source audit reports published/updated date signals present.

TECH-022 — Structured data and canonical integrity

Field	Implementation specification
Source / URL / input	All indexable HTML pages
Why it matters	Checks canonical tags plus JSON-LD validity and consistency.
Exact implementation steps	Extract rel=canonical; parse JSON-LD; validate syntax; compare canonical to final URL; flag conflicts and duplicate canonical tags.
Decision logic	One canonical matching final preferred URL and valid JSON-LD = pass. Conflicts or invalid JSON-LD = partial/fail.
Calculation	Pages passing both canonical and structured-data checks / eligible pages ×100.
Status rules	PASS ≥90; PARTIAL 70-89; FAIL <70.
Evidence to store	Canonical URL(s), schema types, parse errors, page URL.
Recommendation	Fix canonical conflicts and invalid structured data.
Implementation tools	BeautifulSoup/lxml + jsonschema/schema validator where applicable

Evoke example: Source audit reports valid JSON-LD and matching canonical.

Caveat: Schema validation should distinguish syntax validity from semantic eligibility for a search feature.

6. On-Page / Content Parameters — 22

ON-001 — Question-based headings

Field	Implementation specification
Source / URL / input	Rendered/HTML headings across service, solution, and resource pages
Why it matters	Checks whether headings match buyer questions that an assistant could retrieve.
Exact implementation steps	Extract H1-H3; classify question-likeness using deterministic interrogatives plus optional LLM classifier; exclude navigation headings.
Decision logic	A heading is positive if it expresses a customer information need, not merely contains a keyword.
Calculation	Question headings / eligible headings ×100, optionally confidence-weighted.
Status rules	PASS ≥70; PARTIAL 40-69; FAIL <40.
Evidence to store	Heading text, classifier label/confidence, page.
Recommendation	Rewrite key headings as customer questions and retain natural wording.
Implementation tools	BeautifulSoup + NLP + optional LLM

Evoke example: Source audit found 0% buyer-question headings and scored this check as fail.

ON-002 — Self-contained answer below question

Field	Implementation specification
Source / URL / input	Content sections beneath question headings
Why it matters	Checks whether the first answer block directly answers the question without requiring other sections.
Exact implementation steps	For each question heading, capture the first paragraph/answer block; run rule + LLM evaluation for directness and completeness.
Decision logic	Pass if the answer can stand alone and directly answers the heading.
Calculation	Passing question sections / eligible question sections ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50; UNKNOWN if no question headings.
Evidence to store	Question, answer text, evaluator rationale/confidence.
Recommendation	Put a concise, self-contained answer immediately after question headings.
Implementation tools	BeautifulSoup + LLM evaluator

Evoke example: Source audit had no question-style headings and therefore could not score answer blocks.

ON-003 — 40-60 word answer window

Field	Implementation specification
Source / URL / input	Question-answer sections
Why it matters	Checks whether initial answers are concise enough for extraction while preserving meaning.
Exact implementation steps	Count words in the first answer paragraph(s) after each question; exclude lists when configured.
Decision logic	Target 40-60 words. 30-80 acceptable; outside range reduces score.
Calculation	Sections within target / eligible sections ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	Word count per section, answer text.
Recommendation	Tighten answer-first sections to a configurable target window.
Implementation tools	Tokenizer + HTML parser

Evoke example: The source audit explicitly specifies a self-contained 40-60 word answer target.

Caveat: Treat range as guidance, not a hard requirement for every topic.

ON-004 — Core concept definition sentences

Field	Implementation specification
Source / URL / input	Service/solution pages
Why it matters	Checks for explicit definitions of important concepts.
Exact implementation steps	Extract candidate concepts from headings/entities; search for patterns such as “X is”, “X refers to”, “we define X as”; use LLM for ambiguous definitions.
Decision logic	Each core concept should have at least one clear definition sentence.
Calculation	Defined concepts / detected core concepts ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	Concept list, definition sentence, source URL.
Recommendation	Add plain-language definitions near first mention.
Implementation tools	NLP + optional LLM

Evoke example: Source audit reports no clear definition sentences for core concepts.

ON-005 — Conclusion-first / answer-first opening

Field	Implementation specification
Source / URL / input	H1 + first 1-2 content blocks
Why it matters	Checks whether page immediately tells the reader the answer/value before deep detail.
Exact implementation steps	Capture H1 and first ~150 words; classify presence of concise summary/answer.
Decision logic	Pass if first paragraph gives decision-relevant answer or definition before supporting detail.
Calculation	Pass pages / eligible pages ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	First paragraph text, classifier score.
Recommendation	Add an answer-first summary immediately after H1.

Implementation tools	HTML parser + LLM evaluator
----------------------	-----------------------------

Evoke example: Source audit reports no short introductory paragraph directly after H1.

ON-006 — FAQ section presence

Field	Implementation specification
Source / URL / input	Service and solution pages
Why it matters	Checks whether important buyer questions have an explicit FAQ section.
Exact implementation steps	Identify service/solution pages via sitemap, breadcrumb/schema, URL patterns, title and classifier; search for FAQ headings.
Decision logic	FAQ section present and non-empty = pass.
Calculation	Service/solution pages with FAQ / eligible pages ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	URL, FAQ heading, number of questions.
Recommendation	Add useful, non-duplicative FAQs to high-value pages.
Implementation tools	Crawler + page classifier

Evoke example: Source audit reports no FAQ section or FAQPage schema.

ON-007 — FAQ structured data

Field	Implementation specification
Source / URL / input	FAQ pages/sections
Why it matters	Checks valid FAQPage schema where appropriate.
Exact implementation steps	Extract JSON-LD; detect FAQPage; validate syntax and question/answer objects; cross-check visible content.
Decision logic	Valid matching FAQ schema = pass; schema-only/inconsistent = partial.
Calculation	Valid FAQ schema pages / FAQ-eligible pages ×100.
Status rules	PASS ≥90; PARTIAL 60-89; FAIL <60.
Evidence to store	Schema payload, question count, visible-vs-schema diff.
Recommendation	Fix schema mismatches; do not add markup for content not visible to users.
Implementation tools	JSON-LD parser + schema validator

Evoke example: Source audit explicitly checks FAQ section or FAQPage schema.

ON-008 — Comparison/specification tables

Field	Implementation specification
Source / URL / input	Buyer evaluation pages
Why it matters	Checks for useful comparison or specification information in real tables where users evaluate options.
Exact implementation steps	Identify pages with comparison intent via title/headings/entities; inspect semantic tables and table headers.
Decision logic	Pass when relevant comparison pages contain structured comparison/spec details.
Calculation	Eligible pages with relevant table / total eligible pages ×100.

Status rules	PASS >=70; PARTIAL 40-69; FAIL <40.
Evidence to store	Detected comparison intent, tables, headers.
Recommendation	Add decision-oriented comparison/specification tables.
Implementation tools	HTML parser + LLM classification

Evoke example: Source audit reports no comparison/specification tables.

ON-009 — List / numbered-step usefulness

Field	Implementation specification
Source / URL / input	Content pages
Why it matters	Checks whether process-oriented content uses real ordered/unordered lists.
Exact implementation steps	Detect list blocks; classify whether list use matches content type.
Decision logic	Process content with appropriate ordered steps = strong; excessive or missing list formatting reduces score.
Calculation	Relevant sections using suitable list markup / relevant sections x100.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	List counts, context, list type.
Recommendation	Convert procedural content into numbered steps and facts into bullet lists.
Implementation tools	HTML parser + classifier

Evoke example: Source audit reports 43 lists/tables as real markup.

ON-010 — Conversational phrasing

Field	Implementation specification
Source / URL / input	Core content
Why it matters	Checks natural language quality versus keyword-stuffed or mechanically repeated phrases.
Exact implementation steps	Extract main text; compute phrase repetition, unusual n-grams and keyword ratios; optional LLM style evaluation.
Decision logic	Natural phrasing with low abnormal repetition = pass.
Calculation	Combine repetition subscore and LLM naturalness score.
Status rules	PASS >=80; PARTIAL 60-79; FAIL <60.
Evidence to store	Top repeated phrases, density metrics, evaluator score.
Recommendation	Rewrite awkward keyword strings as natural customer language.
Implementation tools	NLP + optional LLM

Evoke example: Source audit reports natural language with no dominant single term.

ON-011 — Keyword stuffing / repetition

Field	Implementation specification
Source / URL / input	Core content on important pages
Why it matters	Detects excessive repeated exact-match terms.
Exact implementation steps	Build term frequency; compare exact-match share and repeated n-grams against site/page baseline; whitelist

	brand/entity names.
Decision logic	Flag when one non-brand term dominates unusually.
Calculation	Penalty-based score: 100 minus configured repetition penalties.
Status rules	PASS ≥ 90 ; PARTIAL 70-89; FAIL < 70 .
Evidence to store	Top terms, counts, density, baseline.
Recommendation	Reduce repetitive phrase use and write semantically rich content.
Implementation tools	NLP / TF-IDF / n-gram analyzer

Evoke example: Do not use a universal keyword-density threshold without benchmarking.

ON-012 — Content depth

Field	Implementation specification
Source / URL / input	Important content pages
Why it matters	Checks whether page content is substantive for its page type.
Exact implementation steps	Count visible words; remove nav/footer boilerplate; compare against template/page-type baseline and intent completeness.
Decision logic	Very low word count plus weak intent coverage = thin. High count alone is not sufficient.
Calculation	Depth score = 50% normalized content coverage + 50% intent coverage.
Status rules	PASS ≥ 80 ; PARTIAL 60-79; FAIL < 60 .
Evidence to store	Unique word count, boilerplate-excluded count, intent coverage.
Recommendation	Expand missing decision information rather than adding filler words.
Implementation tools	BeautifulSoup + text extraction + LLM/topic coverage

Evoke example: Source audit reports an example page with 9,477 words and considers it not thin.

ON-013 — Original / citable data points

Field	Implementation specification
Source / URL / input	Major service/resource pages
Why it matters	Checks for unique statistics, benchmarks, measurements or original findings.
Exact implementation steps	Extract numeric claims; classify whether they appear original and associated with a source or company evidence.
Decision logic	Count distinct useful data points; discount generic dates/prices that are not evidence.
Calculation	Citable data-point coverage score, benchmarked by page type.
Status rules	PASS ≥ 70 ; PARTIAL 40-69; FAIL < 40 .
Evidence to store	Claim text, numeric value, context, source status.
Recommendation	Add original data, benchmarks and case-study metrics.
Implementation tools	NLP + regex + LLM verifier

Evoke example: Source audit found 615 statistic-like data points.

ON-014 — Claims sourced to primary source

Field	Implementation specification
Source / URL / input	Claims with external factual assertions
Why it matters	Checks whether important claims cite or link to primary sources.
Exact implementation steps	Extract sentences with factual numeric/market claims; detect nearby source links or phrases; optionally verify source domain type.
Decision logic	Important claims with primary-source evidence score higher.
Calculation	Cited claims / auditable claims ×100.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	Claim text, source URL, source type, date.
Recommendation	Add source and date to material factual claims.
Implementation tools	NLP + link extraction + domain allowlist

Evoke example: Source audit reports no sourcing phrases and marks this as fail.

ON-015 — Source freshness

Field	Implementation specification
Source / URL / input	Pages with source citations
Why it matters	Checks whether referenced facts are current for their topic.
Exact implementation steps	Parse dates from linked sources; classify topic shelf life; compare age to configurable freshness threshold.
Decision logic	Freshness depends on topic; e.g., fast topics require recent sources, evergreen sources can be older.
Calculation	Fresh cited sources / dated cited sources ×100.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	Source date, retrieval date, topic freshness policy.
Recommendation	Refresh stale citations and dated claims.
Implementation tools	Crawler + date parser + ruleset

Caveat: Do not reject old authoritative sources when the fact itself is historical/evergreen.

ON-016 — Named author

Field	Implementation specification
Source / URL / input	Content pages
Why it matters	Checks whether content identifies a real author or equivalent accountable owner.
Exact implementation steps	Inspect byline, author schema, rel=author, profile link.
Decision logic	At least one credible named author signal = pass.
Calculation	Pages with author signal / eligible pages ×100.
Status rules	PASS >=90; PARTIAL 70-89; FAIL <70.
Evidence to store	Author name, role, profile URL, schema.
Recommendation	Add author attribution and profile pages for content where expertise matters.
Implementation tools	HTML + JSON-LD parser

Evoke example: Source audit found a named author or author schema.

ON-017 — Author credentials / expertise

Field	Implementation specification
Source / URL / input	Author profile and content schema
Why it matters	Checks whether author expertise is described and linked.
Exact implementation steps	Follow author profile; extract role, credentials, organization, expertise; compare with content topic.
Decision logic	Credible role/profile linked and topically relevant = pass.
Calculation	Profiles meeting completeness criteria / profiles checked ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	Author profile fields, credentials, topical relevance score.
Recommendation	Add concise professional credentials and real profile details.
Implementation tools	Crawler + NLP/LLM relevance scorer

Evoke example: Source audit reports named author with role/credentials and a real profile signal.

ON-018 — Reviewer / technical approver

Field	Implementation specification
Source / URL / input	High-stakes or technical content
Why it matters	Checks whether technical content has a named reviewer/approver when appropriate.
Exact implementation steps	Identify content flagged as technical/claims-heavy; inspect “reviewed by”, reviewer schema or visible approval note.
Decision logic	Technical pages with appropriate reviewer = pass; not applicable = N/A.
Calculation	Passing eligible technical pages / eligible technical pages ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50; N/A for non-technical pages.
Evidence to store	Reviewer name, role, profile, date.
Recommendation	Add subject-matter review for technical claims.
Implementation tools	HTML + classifier

Evoke example: Source audit marks reviewer/technical approver as fail.

ON-019 — Customer proof and quantified outcomes

Field	Implementation specification
Source / URL / input	Case studies, service pages, proof sections
Why it matters	Checks for named clients, measurable outcomes and deployment detail.
Exact implementation steps	Extract entity names, percentages, time/cost outcomes, implementation details; verify supporting case-study page.
Decision logic	Named customer + quantified outcome + context = strongest evidence.
Calculation	Evidence-rich pages / eligible pages ×100.
Status rules	PASS ≥70; PARTIAL 40-69; FAIL <40.
Evidence to store	Client name, metric, outcome, evidence URL.
Recommendation	Add case studies with concrete metrics and implementation details.
Implementation tools	NLP + NER + regex + LLM

Evoke example: Source audit reports client/customer references alongside quantified results.

ON-020 — Buying-journey coverage

Field	Implementation specification
Source / URL / input	Sitemap + page content
Why it matters	Checks discovery, qualification, comparison, alternatives, objections and trust coverage.
Exact implementation steps	Classify pages into 6 stages using title/headings/semantic vectors; map pages to stage; identify gaps.
Decision logic	A stage is covered when enough relevant, quality pages exist.
Calculation	Covered stages / 6 ×100.
Status rules	PASS >=83 (5-6 stages); PARTIAL 50-82 (3-4); FAIL <50 (0-2).
Evidence to store	Page-to-stage mapping, gap list, confidence.
Recommendation	Create content for missing stages, especially comparison/objection/trust pages.
Implementation tools	Embeddings + LLM classifier + sitemap

Evoke example: Source audit suggests coverage of 4/6 buying-journey stages.

ON-021 — Search-intent overlap / duplicate questions

Field	Implementation specification
Source / URL / input	All important pages and headings
Why it matters	Checks whether multiple pages compete for the same query intent.
Exact implementation steps	Create embeddings for H1/H2/title + first paragraph; pairwise similarity; compare same-topic clusters; optionally use keyword overlap.
Decision logic	High semantic similarity across different canonical URLs for the same intent = cannibalization candidate.
Calculation	1 - weighted overlap ratio, normalized 0-100.
Status rules	PASS >=90; PARTIAL 70-89; FAIL <70.
Evidence to store	Page pairs, similarity scores, shared phrases, canonical URLs.
Recommendation	Merge, differentiate, or retarget competing pages.
Implementation tools	Sentence-transformers/OpenAI embeddings + clustering

Evoke example: Source audit found duplicate heading text and marked this issue as fail.

Caveat: Do not merge pages solely because they share a keyword; compare intent.

ON-022 — Brand + category + geography association

Field	Implementation specification
Source / URL / input	Home/about/service pages and entity mentions
Why it matters	Checks whether the brand is clearly named alongside its category, industry and geography.
Exact implementation steps	Extract brand name from title/schema; search near-term context for category, industry and location entities; use NER/LLM for semantic association.

Decision logic	All three associations on priority pages = strong.
Calculation	Pages meeting all required associations / priority pages ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	Brand, category terms, industry terms, geography terms, snippets.
Recommendation	State what Evoke Technologies is, whom it serves, and where it operates in clear language.
Implementation tools	NER + rules + LLM verifier

Evoke example: Source audit marks brand/category/industry/geography association as partial.

7. Off-Page / Authority Parameters — 18

OFF-001 — Wikidata presence

Field	Implementation specification
Source / URL / input	Wikidata search/API for exact organization name/domain
Why it matters	Checks whether the organization has a Wikidata entity.
Exact implementation steps	Search by exact organization name and website domain; resolve candidate entity; store entity ID and URL.
Decision logic	Exact organization entity with matching domain = pass.
Calculation	Presence score 0/50/100 plus accuracy subscore.
Status rules	PASS ≥90; PARTIAL 50-89; FAIL <50; UNKNOWN if API unavailable.
Evidence to store	Entity ID, labels, official website property, retrieval time.
Recommendation	Create/maintain a correct entity where appropriate.
Implementation tools	Wikidata API adapter

Evoke example: The source audit marks this illustrative and says live off-page data requires integration.

Caveat: Existence alone is not authority; validate identity before scoring.

OFF-002 — Wikidata accuracy

Field	Implementation specification
Source / URL / input	Matched Wikidata entity
Why it matters	Checks key fields against official site data.
Exact implementation steps	Compare organization name, official website, country/region and other supported properties; avoid writing to Wikidata automatically.
Decision logic	Matching high-confidence identity fields = pass.
Calculation	Matched fields / fields checked ×100.
Status rules	PASS ≥90; PARTIAL 70-89; FAIL <70.
Evidence to store	Field-by-field match matrix.
Recommendation	Correct inconsistent public knowledge data through appropriate channels.
Implementation tools	Wikidata API + normalization

OFF-003 — Knowledge Panel presence

Field	Implementation specification
Source / URL / input	Search result / trusted SERP data provider
Why it matters	Checks whether a branded knowledge panel or entity card appears.
Exact implementation steps	Run branded search via approved SERP data provider or controlled manual verification; record result evidence.
Decision logic	Visible entity panel for correct organization = pass.
Calculation	Presence + correctness score.
Status rules	PASS when verified; PARTIAL when ambiguous; UNKNOWN without a supported SERP source.
Evidence to store	SERP snapshot metadata, provider, query, timestamp.
Recommendation	Improve entity consistency and authoritative third-party references.
Implementation tools	SERP API/provider adapter

Evoke example: Source audit marks this illustrative rather than live-checked.

Caveat: Do not scrape search pages in ways that violate provider terms.

OFF-004 — LinkedIn company profile completeness

Field	Implementation specification
Source / URL / input	LinkedIn company profile / licensed data source
Why it matters	Checks official company profile presence and key fields.
Exact implementation steps	Resolve official profile; verify name, website, description, industry and locations where data is available.
Decision logic	Completeness checklist per field.
Calculation	Matched/available fields / required fields ×100.
Status rules	PASS >=90; PARTIAL 60-89; FAIL <60; UNKNOWN without data source.
Evidence to store	Profile URL and checked fields.
Recommendation	Keep official organization information consistent.
Implementation tools	Approved provider/API or manual import

Evoke example: Availability depends on data licensing; do not implement scraping blindly.

OFF-005 — Crunchbase/company database presence

Field	Implementation specification
Source / URL / input	Licensed business-data source
Why it matters	Checks presence and basic consistency in a recognized business database.
Exact implementation steps	Resolve entity; compare name, website, category, locations, founding details if present.
Decision logic	Correct entity + sufficient completeness = pass.
Calculation	Completeness score.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50; UNKNOWN if unavailable.
Evidence to store	Provider, entity ID, fields, retrieval timestamp.
Recommendation	Complete authoritative business profiles.
Implementation tools	Provider adapter

OFF-006 — Business directory consistency

Field	Implementation specification
Source / URL / input	Configured list of relevant business/industry directories
Why it matters	Checks whether organization data is consistent across selected authoritative directories.
Exact implementation steps	Define source list; retrieve records; normalize name/address/domain/phone if available; compare fields.
Decision logic	Exact normalized match across core fields = consistent.
Calculation	Matching field values / total comparable field values ×100.
Status rules	PASS >=90; PARTIAL 70-89; FAIL <70.
Evidence to store	Per-source values and normalized comparison matrix.
Recommendation	Correct inconsistent public organization information.
Implementation tools	Adapter layer + normalization

OFF-007 — Company NAP/entity consistency

Field	Implementation specification
Source / URL / input	Official site vs external profiles
Why it matters	Checks name/address/contact/domain consistency across sources.
Exact implementation steps	Use normalized canonical values; compare each external record to official site.
Decision logic	Exact normalized name/domain/address = match; minor formatting differences ignored.
Calculation	Matched sources / sources checked ×100.
Status rules	PASS >=90; PARTIAL 70-89; FAIL <70.
Evidence to store	Source, raw value, normalized value, mismatch type.
Recommendation	Standardize organization details across external profiles.
Implementation tools	Normalization library + adapters

Evoke example: Source audit asks for identical company name/address/details across sources.

Caveat: For enterprise organizations, allow multiple legitimate office locations.

OFF-008 — Certification verification

Field	Implementation specification
Source / URL / input	Official certification/partner pages and recognized issuers
Why it matters	Checks whether claimed certifications/tiers can be verified off-site.
Exact implementation steps	Extract certification claims from site; identify issuing body; search official issuer directory/API; compare legal/entity name.
Decision logic	Verified issuer record = pass. Claim without verification = partial/fail.
Calculation	Verified claims / claims checked ×100.
Status rules	PASS >=90; PARTIAL 50-89; FAIL <50; UNKNOWN if issuer data inaccessible.
Evidence to store	Claim URL, issuer URL, credential ID if public, verification date.
Recommendation	Link claims to official verification pages and keep claims

	current.
Implementation tools	Web adapter + issuer APIs where available

Evoke example: Source audit explicitly lists CMMI/ISO/partner tiers as illustrative checks.

OFF-009 — G2 profile completeness

Field	Implementation specification
Source / URL / input	G2 organization profile / licensed data
Why it matters	Checks presence and completeness of customer review profile.
Exact implementation steps	Resolve profile; inspect company description, category, services, website, logo and review count if licensed data exists.
Decision logic	Completeness checklist.
Calculation	Completed profile fields / required fields ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50; UNKNOWN if unavailable.
Evidence to store	Profile URL, fields, review count/date if available.
Recommendation	Complete authoritative profile and keep service/category mapping accurate.
Implementation tools	Approved API/data source

OFF-010 — Clutch profile completeness

Field	Implementation specification
Source / URL / input	Clutch profile / licensed data
Why it matters	Checks profile presence and service/category/location consistency.
Exact implementation steps	Resolve official profile; compare service categories, website, location and company name.
Decision logic	Correct identity and complete core fields = pass.
Calculation	Fields matched / fields required ×100.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50.
Evidence to store	Profile URL, comparison fields.
Recommendation	Complete and normalize agency profile information.
Implementation tools	Approved data source

OFF-011 — Gartner/TrustRadius presence

Field	Implementation specification
Source / URL / input	Selected review/analyst data source
Why it matters	Checks whether relevant profiles exist and contain consistent positioning.
Exact implementation steps	Search supported source; resolve entity; inspect presence/completeness if data permitted.
Decision logic	Presence + completeness score.
Calculation	Weighted presence/completeness.
Status rules	PASS ≥80; PARTIAL 50-79; FAIL <50; UNKNOWN if no supported data.
Evidence to store	Provider, entity, fields, timestamp.
Recommendation	Claim/complete appropriate industry profiles.
Implementation tools	Provider adapter

OFF-012 — Review volume

Field	Implementation specification
Source / URL / input	Configured review platforms
Why it matters	Measures number of reviews relative to a defined competitor benchmark.
Exact implementation steps	Collect review counts from supported platforms; define comparison cohort; normalize by platform.
Decision logic	Volume index = company reviews / median competitor reviews, capped for scoring.
Calculation	Score = normalized index 0-100.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	Review counts, platform, competitor cohort, retrieval time.
Recommendation	Build sustained review acquisition across relevant platforms.
Implementation tools	Review platform adapters

Evoke example: Source audit identifies review volume as illustrative and requires data-source integrations.

Caveat: Do not fabricate or incentivize misleading reviews.

OFF-013 — Review recency

Field	Implementation specification
Source / URL / input	Configured review platforms
Why it matters	Measures how recently reviews were posted.
Exact implementation steps	Collect review dates from supported sources; calculate median/weighted recency.
Decision logic	Newer reviews increase score, topic-specific shelf life applies.
Calculation	Recency score = weighted freshness function.
Status rules	PASS >=80 if recent; PARTIAL 50-79; FAIL <50.
Evidence to store	Latest review dates, median age, review count.
Recommendation	Maintain continuous, authentic review activity.
Implementation tools	Review adapters

OFF-014 — Review velocity

Field	Implementation specification
Source / URL / input	Configured review platforms + historical snapshots
Why it matters	Measures rate of new reviews over time.
Exact implementation steps	Store review count snapshots; compute delta per 30/90 days.
Decision logic	Velocity = (current_count - prior_count) / days × normalized factor.
Calculation	Compare to competitor median or site baseline.
Status rules	PASS >=80 vs benchmark; PARTIAL 50-79; FAIL <50.
Evidence to store	Snapshots, interval, delta, rate.
Recommendation	Create a steady review-generation process.
Implementation tools	Review adapters + time-series DB

Evoke example: Requires historical scan data or provider history.

OFF-015 — Backlink quality and topical relevance

Field	Implementation specification
Source / URL / input	Backlink API/provider
Why it matters	Measures referring domains, authority and topical fit.
Exact implementation steps	Retrieve backlink dataset; deduplicate referring domains; exclude spam/toxic sources with configurable rules; score authority and topical relevance.
Decision logic	Weighted domain score = authority × relevance × link uniqueness; aggregate using diminishing returns.
Calculation	Example: backlink_quality = mean(top-N weighted domain scores).
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50; UNKNOWN without provider.
Evidence to store	Referring domain, authority metrics, topical category, link URL, first/last seen.
Recommendation	Prioritize earning relevant, reputable industry links rather than raw volume.
Implementation tools	Backlink provider adapter

Evoke example: Source audit marks quality/topical relevance of links as illustrative fail.

Caveat: Do not equate third-party domain score with Google ranking impact.

OFF-016 — Unlinked brand mentions and earned news

Field	Implementation specification
Source / URL / input	Search/news/mention data provider
Why it matters	Finds public mentions of the company without clickable links and measures earned coverage.
Exact implementation steps	Query exact brand, variants and domain; dedupe URLs; classify news/industry/content; detect whether domain link exists.
Decision logic	Unlinked mention = brand mention where target domain absent from outbound links.
Calculation	Opportunity score based on relevant unlinked mentions + earned-coverage count + recency.
Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	Mention URL, date, source type, linked/unlinked, relevance.
Recommendation	Prioritize legitimate outreach/reclamation opportunities and earned media.
Implementation tools	Search/news provider + parser

OFF-017 — Third-party pages and “best/top/alternatives” coverage

Field	Implementation specification
Source / URL / input	Search/SERP or web-index provider
Why it matters	Checks whether relevant third-party pages mention the brand for category/comparison/alternative queries.
Exact implementation steps	Generate a configured query set; retrieve supported SERP/web-index results; classify list/comparison pages; detect brand mentions.
Decision logic	Mention in relevant high-authority pages = positive.
Calculation	Relevant pages mentioning brand / relevant pages discovered ×100.

Status rules	PASS >=80; PARTIAL 50-79; FAIL <50.
Evidence to store	Query, result URL, rank, page type, brand presence, date.
Recommendation	Pursue legitimate inclusion through partnerships, PR, product differentiation and evidence.
Implementation tools	SERP/web-index adapter + LLM page classifier

Evoke example: Source audit includes “best/top/leading” and “alternatives to competitor” coverage.

Caveat: Do not attempt paid/manipulative inclusion solely to change this metric.

OFF-018 — Analyst, directory, trade-press and citation-source coverage

Field	Implementation specification
Source / URL / input	Configured authoritative third-party sources
Why it matters	Measures quality of external references that describe the organization accurately.
Exact implementation steps	Build source taxonomy; discover relevant pages; score source authority, relevance and description accuracy; dedupe.
Decision logic	Weighted coverage = $\Sigma(\text{source_weight} \times \text{accurate_mention}) / \Sigma(\text{source_weight})$.
Calculation	PASS >=80; PARTIAL 50-79; FAIL <50; UNKNOWN if discovery source unavailable.
Status rules	Source URL, source type, authority bucket, mention accuracy, date.
Evidence to store	Increase accurate, authoritative third-party coverage and correct factual errors.
Recommendation	Search/web index + LLM accuracy classifier
Implementation tools	Source audit marks analyst/directory/trade-press coverage as illustrative fail and also checks description accuracy.

Evoke example: Separate source discovery from authority scoring so provider bias can be audited.

8. Parameter Registry Design

```
parameter_id: TECH-001
name: AI crawler permissions
section: technical
weight: 1.0
source_type: website
input: robots.txt
validator: robots_ai_crawler
score_range: 0-100
status_bands: {pass: 90, partial: 60}
unknown_policy: exclude
evidence_schema: [checked_url, crawler, rule, decision]
```

Config field	Meaning
parameter_id	Stable ID that never changes once published
source_type	website / api / search / provider / ai_evaluator
validator	Code function or service that performs the check

weight	Importance in category scoring
thresholds	Configurable pass/partial/fail bands
unknown_policy	exclude, zero, or report-only; recommended: exclude
evidence_schema	Structured fields needed for UI and auditing

9. Scan Execution Order

1. Normalize user input: https/http, www/non-www, trailing slash, IDN and redirects.
2. Fetch robots.txt first; discover sitemap URLs from robots and common sitemap locations.
3. Fetch and recursively parse sitemap indexes; store candidate URLs.
4. Crawl homepage and high-priority pages; discover additional internal links within configured crawl budget.
5. Run deterministic technical checks on fetched HTML and HTTP metadata.
6. Run content extraction and page-type classification.
7. Run On-Page deterministic rules first; run LLM evaluation only on the smaller set of ambiguous/semantic checks.
8. Run Off-Page adapters only when configured credentials/data sources are available.
9. Persist evidence and parameter results before calculating category scores.
10. Calculate category scores, overall score, coverage score and prioritized issues.
11. Generate dashboard JSON and exportable report.

10. Crawl Policy for evoketechnologies.com

Setting	Recommended V1
Starting URLs	Homepage + sitemap URLs
Maximum pages	Configurable; e.g. 250 for demo, 1,000 for production tier
Concurrency	5-10 concurrent requests per host
Timeout	10-15 seconds per request
Retries	2 retries with exponential backoff for transient errors
Allowed content	text/html, application/xhtml+xml, XML sitemap, robots.txt, JSON-LD extracted from HTML
Skip	Binary downloads, large media, mailto/tel/javascript, tracking URLs, query duplicates unless required
Politeness	Respect robots rules for your own crawler; use a clear audit user-agent
Rendering	Use Playwright only for pages where raw HTML cannot provide enough evidence
Caching	Cache URL responses by hash during a scan; optionally persist across scans with TTL

11. LLM / AI Evaluator Design

Use deterministic rules wherever a machine-readable signal exists. Use an LLM only for semantic judgments such as question intent, directness, definition quality, buyer-journey stage, or description accuracy. Every LLM result should include a confidence score and the exact text span that was evaluated.

Input -> evidence text + parameter rubric -> LLM -> JSON {label, score, confidence, rationale, evidence_span}

Reject malformed JSON -> retry once -> if still invalid, mark UNKNOWN and log evaluator error.

12. External Data / Off-Page Adapter Layer

Adapter	Required capabilities	Failure behavior
Backlink provider	Domains, links, authority/relevance fields, timestamps	Mark OFF-015 UNKNOWN; never score missing data as zero
Review provider	Profiles, review counts/dates, historical counts if supported	Mark OFF-009/10/11/12/13/14 according to unavailable source
SERP/web index	Query results, URLs, rank, snippets, timestamps where supported	Mark discovery parameters UNKNOWN if no licensed source
Business profile source	Organization profiles and fields	Mark consistency checks UNKNOWN
Certification/issuer source	Verification result	Mark certification verification UNKNOWN

13. Recommendation / Priority Engine

Priority Score = severity_weight × score_gap × page_impact × effort_factor

Severity	Default interpretation
CRITICAL	Blocks important discoverability or causes widespread trust/crawl failure
HIGH	Materially reduces AI/content visibility on priority pages
MEDIUM	Meaningful quality/coverage improvement but not blocking
LOW	Backlog optimization or external authority improvement

The uploaded audit itself uses Critical / Medium / Low buckets. Its critical examples include FAQ sections, buyer-question headings, clear definitions, answer-first structure, primary sourcing, reviewer signals and avoiding duplicate intent. Its medium list includes Core Web Vitals, brand/category/geography association, image alt text/transcripts, third-party citation sources and comparison tables.

14. Dashboard Data Contract

```
{
  "domain": "www.evoketechnologies.com",
  "overall_score": 69,
  "category_scores": {"technical": 89, "on_page": 60, "off_page": 54},
  "status_counts": {"pass": 0, "partial": 0, "fail": 0, "unknown": 0},
  "parameters": [],
  "top_issues": [],
  "coverage": {"scorable_parameters": 62, "known": 62}
}
```

Dashboard screen	Required data
Overview	Overall score, category scores, status counts, top 5 issues, scan timestamp, coverage
Technical	22 results, URL-level evidence, crawl stats, performance metrics
On-Page	22 results, question/answer examples, content coverage, intent clusters
Off-Page	18 results, provider coverage, external profile matrix, mention/review/backlink evidence
Parameter detail	Single parameter score, evidence, affected URLs, formula, recommendation
Fix queue	Severity, effort, affected pages, expected score impact
History	Score trend by scan, parameter movement, resolved issues

15. QA and Validation Plan

12. Unit-test every deterministic parser with synthetic fixtures: robots.txt, sitemap XML, HTML headings, redirects, JSON-LD, image alt text and canonical variations.
13. Create regression fixtures for allowed and blocked AI crawlers, including wildcard rules and conflicting user-agent groups.
14. Test crawl behavior on 200/301/302/404/410/429/500 responses and redirect loops.
15. Compare raw HTML word counts to rendered counts on static and JavaScript-heavy pages.
16. For LLM checks, keep a gold dataset of manually labeled headings, answers, definitions and buying-journey pages.
17. For Off-Page adapters, mock API responses and verify UNKNOWN behavior when credentials or data are unavailable.
18. Verify scoring with known inputs where the expected overall score is deterministic.
19. Store parser/evaluator version with every scan so historical scores remain explainable.

16. Evoke Technologies Baseline Mapping from the Uploaded Audit

Source audit signal	Reported value	Where it maps in new engine
Overall score	69/100	Overall scoring engine
Technical health	89/100	TECH-001 to TECH-022
Content & answers	60/100	ON-001 to ON-022
Reputation & authority	54/100	OFF-001 to OFF-018
Sitemap	201 URLs	TECH-005 evidence
Performance estimate	23/100	TECH-008 evidence
Image alt text	60%	TECH-020 evidence
Lists/tables	43 lists, 0 tables	TECH-018 and ON-008/009 evidence
Core content	9477 words on sampled page	ON-012 evidence
Internal links	210	TECH-006 crawl evidence
Buyer journey	4/6 stages	ON-020 evidence

17. Recommended Delivery Roadmap

Phase	Scope	Outcome
Phase 1	TECH-001 to TECH-022 + ON-001/003/006/007/008/009/011/012/014/016/021/022	Fully automated crawl/content MVP
Phase 2	Remaining On-Page semantic checks + LLM evaluator	High-quality content/AI-readiness scoring
Phase 3	Off-Page adapters with one provider per data type	Live authority/reputation checks
Phase 4	AI assistant visibility and market modules	Expand beyond 62 core parameters
Phase 5	Historical trend + benchmark + multi-domain reports	Productization for multiple customers

18. Non-Negotiable Implementation Rules

- Every parameter must have a stable ID and a machine-readable rule.
- Every scored result must have evidence; no score without evidence.
- Every UNKNOWN result must state exactly which dependency or data source was unavailable.
- Never silently treat missing API data as FAIL.
- Store raw inputs plus normalized outputs so a finding can be reproduced.
- Separate “detected” from “recommended”: detection should be objective; recommendation can be generated from templates or LLMs.
- Version thresholds and scoring weights; never hard-code them in dashboard code.
- Use the same evidence object for API, dashboard and PDF export so the three outputs cannot disagree.
- For external-source parameters, show provider name and retrieval time in the UI.
- Allow a parameter to be marked Not Applicable when the rule genuinely does not apply to a page type; do not punish a site for missing an irrelevant feature.

19. Cursor Implementation Acceptance Criteria

The build is complete only when: (1) a user can enter <https://www.evoketechnologies.com/> on Page 1 and start a scan; (2) one scan_id is created and persisted; (3) all 62 parameter jobs are attempted independently; (4) Page 1 shows live or polling progress by section; (5) Page 2 renders only persisted scan results; (6) every parameter has evidence/status/score or UNKNOWN with an error reason; (7) category scores and the overall score are server-generated; (8) the Download Report action produces a PDF containing the same persisted results shown on Page 2; and (9) re-running the same URL creates a new scan rather than overwriting the previous scan.

19.1 Definition of Done for a Parameter

A parameter is considered implemented only when its source is defined, request/fetch method is defined, parsing/extraction is implemented, pass/partial/fail/unknown logic is implemented, score calculation is implemented, evidence is persisted, recommendation generation is implemented, timeout/error handling is implemented, and at least one positive and one negative test fixture exists.

Appendix A — Minimum Python Service Layout

```
app/  
  api/  
    scans.py  
    reports.py  
  crawler/  
    robots.py  
    sitemap.py  
    http.py  
    links.py  
    render.py  
  parameters/  
    registry.yaml  
    technical.py  
    onpage.py  
    offpage.py  
  evaluators/  
    deterministic.py  
    llm.py  
  scoring/  
    category.py  
    overall.py  
    priority.py  
  storage/  
    models.py  
    repository.py  
  adapters/  
    backlinks.py  
    reviews.py  
    serp.py  
    business_profiles.py
```

Appendix B — Example Parameter Result

```
{  
  "parameter_id": "TECH-001",  
  "domain": "www.evoketechnologies.com",  
  "checked_url": "https://www.evoketechnologies.com/robots.txt",  
  "score": 100,  
  "status": "PASS",  
  "evidence": {  
    "http_status": 200,  
    "crawlers": [  
      {"name": "GPTBot", "decision": "ALLOWED", "rule": "User-agent: GPTBot / Disallow: "},  
      {"name": "ClaudeBot", "decision": "ALLOWED"}  
    ]  
  },  
  "recommendation": null,  
  "checked_at": "2026-08-18T...Z",  
  "engine_version": "1.0.0"  
}
```